



سوالات نظری

سوال ۱:

(آ) نادرست؛ در جستجوی گرافی تمام گره‌هایی که قبلاً از آنها بازدید کرده‌ایم نیز ذخیره می‌شوند تا از بررسی مجدد آنها جلوگیری شود. بنابراین پیچیدگی فضایی^۱ الگوریتم DFS در این حالت از مرتبه $O(b^m)$ می‌باشد.

(ب) درست؛ طبق تعریف تابع اکتشافی h یک‌نوا است اگر:

$$\forall A, B : h(A) - h(B) \leq \text{cost}(A \rightarrow B)$$

حال اگر دو تابع اکتشافی h_1 و h_2 یک‌نوا باشند، به سادگی می‌توان نتیجه گرفت که:

$$h_1(A) - h_1(B) \leq \text{cost}(A \rightarrow B)$$

$$h_2(A) - h_2(B) \leq \text{cost}(A \rightarrow B)$$

با جمع دو رابطه بالا داریم:

$$(h_1 + h_2)(A) - (h_1 + h_2)(B) \leq 2 \times \text{cost}(A \rightarrow B)$$

با تقسیم طرفین بر ۲ نتیجه می‌شود:

$$\forall A, B : \left(\frac{h_1 + h_2}{2} \right) (A) - \left(\frac{h_1 + h_2}{2} \right) (B) \leq \text{cost}(A \rightarrow B)$$

بنابراین $\frac{h_1 + h_2}{2}$ یک تابع اکتشافی یک‌نوا است.

(ج) نادرست؛ مثال نقض می‌آوریم. فرض کنید به ازای تابع اکتشافی h_1 و دو گره A و B داریم:

$$h_1(A) = 3$$

$$h_1(B) = 2$$

$$\text{cost}(A \rightarrow B) = 2$$

می‌دانیم که h_1 یک‌نوا هست و رابطه $h_1(A) - h_1(B) \leq \text{cost}(A \rightarrow B)$ صحیح می‌باشد. حال اگر عدد ثابت $c = 3$ را در نظر بگیریم:

$$h_2(A) = 9$$

$$h_2(B) = 6$$

اکنون داریم $h_2(A) - h_2(B) = 3 \not\leq \text{cost}(A \rightarrow B)$ و رابطه مورد نظر دیگر برای گره‌های A و B صادق نیست، پس تابع h_2 یک‌نوا نیست.

(د) درست: می‌دانیم طبق تعریف به تابع اکتشافی‌ای قابل قبول^۲ گفته می‌شود که:

$$\forall n, 0 \leq h(n) \leq h^*(n)$$

که $h^*(n)$ هزینه واقعی است.

اثبات. فرض کنید دو هدف قابل دسترس A و B داریم که A بهینه و B غیر بهینه^۲ است. n یک جد دلخواه A (شامل خود A) است؛ می‌خواهیم نشان دهیم که n زودتر از B ، از fringe خارج می‌شود. از سه مورد زیر استفاده می‌کنیم:

۱. $g(A) < g(B)$ ؛ چون A هدف بهینه، و B غیر بهینه است.

۲. $h(A) = h(B) = 0$ ؛ چون تابع h قابل قبول است.

۳. $f(n) \leq f(A)$ ؛ به این شکل نشان داده می‌شود:

$$\begin{aligned} h(n) \leq h^*(n) &\Rightarrow g(n) + h(n) \leq g(n) + h^*(n) \\ &\Rightarrow f(n) \leq g(A) \\ &\xRightarrow{h(A)=0} f(n) \leq f(A) \end{aligned}$$

اکنون از این سه مورد استفاده می‌کنیم:

$$\begin{aligned} 2 \text{ و } 1 &\Rightarrow g(A) + h(A) < g(B) + h(B) \Rightarrow f(A) < f(B) \\ &\xRightarrow{3} f(n) < f(B) \quad (*) \end{aligned}$$

چون از ابتدا فرض کرده بودیم که n یک جد دلخواه A (شامل خود A) هست، طبق رابطه $(*)$ نتیجه می‌گیریم که تمام اجداد A از جمله خودش، پیش از B بسط داده می‌شوند؛ پس نتیجه می‌گیریم که در این حالت الگوریتم A^* بهینه است. $\square$

(ه) نادرست؛ الگوریتم Local Beam Search با $k = 1$ ، همواره تمام همسایه‌های حالت فعلی را تولید کرده و سپس بهترین گزینه بین آنها را انتخاب می‌کند؛ بنابراین اگر تمام همسایه‌ها بدتر از حالت فعلی باشند، الگوریتم به ناچار بهترین آنها را (که نسبت به حالت فعلی بدتر است) انتخاب می‌کند. در مقابل الگوریتم Simulated Annealing هر بار به طور تصادفی یکی از همسایه‌ها را انتخاب می‌کند: اگر این همسایه بهتر از حالت فعلی باشد، انتخاب می‌شود. اما اگر بدتر باشد، با احتمالی انتخاب می‌شود که این احتمال با کاهش دما کم می‌شود. به طور خلاصه هنگامی که دما به صفر میل می‌کند، الگوریتم SA همسایه‌ای که بدتر از حالت فعلی باشد را انتخاب نمی‌کند، و از طرفی همسایه انتخاب شده هم لزوماً «بهترین» همسایه ممکن نیست.

(و) نادرست؛ الگوریتم BFS در طول اجرای خود هر گره را حداکثر یک بار بسط می‌دهد. در مقابل، الگوریتم IDS هر بار که حد عمق (limit) جدیدی تعریف می‌کند، جستجو را مجدداً از ریشه درخت آغاز کرده و تمام گره‌های سطوح قبلی را دوباره بسط می‌دهد. به همین دلیل، گره‌های سطوح بالاتر چندین بار (به عنوان مثال، گره ریشه $1 + d$ بار) باز می‌شوند. در نتیجه، تعداد کل گره‌های بسط‌یافته در IDS همواره اکیداً بیشتر از BFS است (هرچند مرتبه مجانبی هر دو $O(b^d)$ می‌باشد).

^۲Admissible
^۳suboptimal

(ز) درست؛ می‌دانیم که الگوریتم UCS همواره گره با کمترین هزینه را از fringe خارج می‌کند. حال اگر یال با هزینه ۰ داشته باشیم، ممکن است الگوریتم در یک حلقه گیر کرده و هیچ گاه خاتمه نیابد. به عنوان مثال فرض کنید هم از گره A به گره B یال داریم و هم از گره B به گره A ، و هزینه هر دو یال نیز صفر است. همچنین فرض کنید در این مسئله هزینه تمام اکشن‌های دیگر بیشتر از صفر باشد. در این صورت اگر الگوریتم به یکی از این دو گره برسد، همواره یال با هزینه صفر را انتخاب می‌کند و هیچ گاه از این حلقه خارج نمی‌شود.

البته باید توجه کنیم که این مثال برای UCS در حالت جستجوی درختی درست است. اگر بخواهیم از الگوریتم در جستجوی گرافی استفاده کنیم، از آنجا که در این حالت الگوریتم لیست گره‌های بسط داده شده را نگه می‌دارد (closed set) امکان ندارد که در دوری با تعداد گره محدود گیر کند؛ در این روش تنها حالتی که امکان دارد گره در یک دور گیر کند، این است که یک دنباله از گره‌ها داشته باشیم که هزینه بین هر دو گره متوالی ۰ باشد؛ در این صورت دیگر فضای مسئله متناهی نیست. (این موضوع منافاتی با صورت مسئله ندارد، چون ممکن است که در این فضا یک پاسخ متناهی داشته باشیم.)

(ح) درست؛ برای اثبات یک‌نوا بودن تابع اکتشافی h باید نشان دهیم به ازای دو گره دلخواه A و B ، که از A به B مسیری وجود داشته باشد:

$$h(A) - h(B) \leq \text{cost}(A \rightarrow B)$$

یا به طور معادل:

$$h(A) \leq \text{cost}(A \rightarrow B) + h(B)$$

اثبات. از یک‌نوا بودن h_1 و h_2 نتیجه می‌گیریم که به ازای هر دو گره دلخواه A و B ، که از A به B مسیری وجود داشته باشد، داریم:

$$h_1(A) \leq \text{cost}(A \rightarrow B) + h_1(B) \quad (۱)$$

$$h_2(A) \leq \text{cost}(A \rightarrow B) + h_2(B) \quad (۲)$$

تابع اکتشافی $h = \min(h_1, h_2)$ را بررسی می‌کنیم. می‌دانیم که:

$$h(B) = h_1(B) \vee h(B) = h_2(B) \quad (۳)$$

$$h(A) \leq h_1(A) \wedge h(A) \leq h_2(A) \quad (۴)$$

بنابراین:

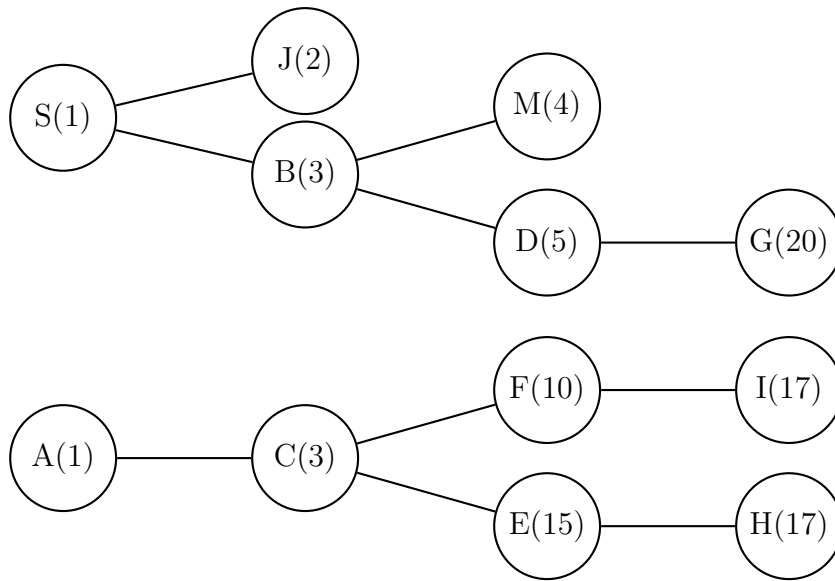
$$(۴) \xrightarrow{(۲), (۱)} \begin{cases} h(A) \leq \text{cost}(A \rightarrow B) + h_1(B) \\ h(A) \leq \text{cost}(A \rightarrow B) + h_2(B) \end{cases} \xrightarrow{(۳)} h(A) \leq \text{cost}(A \rightarrow B) + h(B)$$

□

بنابراین تابع اکتشافی h یک‌نوا است.

(ط) نادرست: به طور کلی با افزایش k در الگوریتم Local Beam Search حالت‌های بیشتری بررسی می‌شوند و شانس گیر کردن در نقاط local maximum یا نواحی flat کمتر می‌شود. اما با افزایش k ، تعداد همسایه‌هایی که باید در هر حالت محاسبه شوند نیز بیشتر می‌شود که در این حالت اجرای هر مرحله از الگوریتم مدت زمان بیشتری طول می‌کشد و ممکن است پیش از رسیدن به حالت هدف، زمان اجرای الگوریتم تمام شود. همچنین در هر مرحله باید همسایه‌های k حالت فعلی ذخیره شوند تا بتوانیم k تای بهتر را از بین آنها انتخاب کنیم. اگر تعداد همسایه‌ها خیلی زیاد شود، ممکن است به محدودیت حافظه بخوریم و الگوریتم متوقف شود.

(ی) نادرست: مثال نقض می‌آوریم. به این شکل توجه کنید:



در این مثال هر گره، یکی از حالت‌های کامل^۴ مسئله است و مقابل اسم هر یک، مقدار تابع fitness به ازای آن نوشته شده. همچنین حالت‌های همسایه با یال به هم متصل شده‌اند. هدف، پیدا کردن حالت با بیشینه مقدار تابع fitness است که در این مثال همان حالت G می‌باشد.

در این مثال، الگوریتم Hill Climbing با شروع از حالت S ، به ترتیب به گره‌های B ، D و در نهایت به حالت هدف G می‌رسد. الگوریتم Local Beam Search با $k = 2$ که از حالت‌های S و A شروع می‌کند، در مرحله اول گره‌های B و C ، سپس گره‌های F و E ، و در نهایت حالت‌های I و H را انتخاب کرده و بدون رسیدن به حالت هدف G ، خاتمه می‌یابد.

سوال ۲:

(آ) توجه: بر اساس الگوریتم TREE-SEARCH موجود در اسلایدهای درس و کتاب راسل و نورویگ فرض می‌کنیم هر گرهی که از fringe خارج می‌شود، اگر هدف نباشد بسط داده می‌شود، حتی اگر خودش برگ باشد و فرزندی نداشته باشد. چون درخت کامل است، می‌دانیم که تعداد گره‌ها در عمق دلخواه x برابر با b^x بوده و تعداد کل گره‌های درخت نیز از $O(b^d)$ است. حال هر الگوریتم را جداگانه بررسی می‌کنیم:

• الگوریتم DFS:

بهترین حالت: می‌دانیم که DFS یک مسیر را انتخاب کرده و آن را به صورت عمقی تا جایی ادامه می‌دهد که به هدف یا بن‌بست بخورد. در بهترین حالت، این الگوریتم در همان ابتدا مسیر درست را انتخاب کرده و به هدف می‌رسد. بنابراین m گره را بسط می‌دهد. (خود هدف به عنوان گره $(m+1)$ ام از fringe خارج می‌شود و چون هدف است دیگر بسط داده نمی‌شود.) این جواب از $O(m)$ است.

بدترین حالت: در بدترین حالت، DFS تمام مسیرهای دیگر را تا انتها (عمق d) پیش می‌رود و در انتها مسیر درست را انتخاب می‌کند. پس در این حالت، الگوریتم تمام گره‌های درخت به جز خود گره هدف و زیردرخت آن را بسط می‌دهد. در این صورت تعداد کل گره‌های بسط داده شده برابر است با:

$$\frac{b^{d+1} - 1}{b - 1} - \frac{b^{d-m+1} - 1}{b - 1} = \frac{b^{d+1} - b^{d-m+1}}{b - 1}$$

که جواب از $O(b^d)$ می‌باشد.

• الگوریتم BFS:

بهترین حالت: BFS همواره کم عمق‌ترین عضو را از fringe خارج می‌کند؛ در واقع می‌توانیم بگوییم BFS به صورت لایه‌ای پیش می‌رود: یعنی ابتدا تمام گره‌ها با عمق ۱ را بررسی می‌کند، سپس تمام گره‌ها با عمق ۲ و ... بنابراین در بهترین حالت، اولین گره با عمق m که از fringe خارج می‌شود همان هدف است. در این حالت، تعداد گره‌های بسط داده شده برابر با یک درخت کامل با ضریب انشعابی b و عمق $m-1$ است که تعداد گره‌های آن با تصاعد هندسی محاسبه شده و برابر با $\frac{b^m - 1}{b - 1}$ است. جواب از $O(b^{m-1})$ است.

بدترین حالت: در این حالت، هدف آخرین گره از لایه با عمق m است که توسط BFS از fringe خارج می‌شود. بنابراین علاوه بر تعدادی که در مورد قبل محاسبه کردیم، $b^m - 1$ عضو دیگر نیز بسط داده می‌شود و تعداد کل گره‌های بسط داده شده در این حالت برابر است با:

$$\frac{b^m - 1}{b - 1} + b^m - 1 = \frac{b^{m+1} - b}{b - 1}$$

که جواب از $O(b^m)$ می‌باشد.

(ب) فرض می‌کنیم در هر دو الگوریتم BFS و DFS هنگامی که به گره‌هایی با اولویت یکسان می‌رسند، گره چپ‌تر را زودتر انتخاب می‌کنند.

• چپ‌ترین گره:

این همان بهترین حالت DFS است که در بخش قبل محاسبه کردیم؛ الگوریتم از ریشه درخت شروع کرده و مستقیم به سمت گره هدف می‌رود. در کل $m+1$ گره از fringe خارج می‌شوند.

BFS تمام گره‌های لایه‌های بالاتر را بررسی می‌کند تا به گره هدف برسد. یعنی در کل $1 + \frac{b^m - 1}{b - 1}$ گره از fringe خارج می‌شود.

در این حالت، عملکرد الگوریتم DFS به مراتب بهتر است.

• **راست‌ترین گره:**

این روش بدترین حالت DFS است و مطابق بخش (آ) تعداد گره‌های خارج شده از $O(b^d)$ می‌باشد.

در این حالت گره هدف آخرین گره از لایه به عمق m است که از fringe خارج می‌شود. در این حالت تعداد کل گره‌های خارج شده، برابر $\frac{b^{m+1} - 1}{b - 1}$ است که از $O(b^m)$ می‌باشد.

در این حالت در صورتی که هدف در لایه آخر نباشد ($m < d$) الگوریتم BFS عملکرد بهتری دارد.

• **حالت متوسط:**

می‌خواهیم امید ریاضی تعداد گره‌های خارج شده از fringe (شامل خود هدف) تا زمان رسیدن به هدف را محاسبه کنیم. برای سادگی، می‌توانیم احتمال خارج شدن هر گره را محاسبه کرده و با هم جمع بزنیم. برای این کار، گره‌ها را بر اساس عمق آنها (h) به دو دسته تقسیم می‌کنیم:

$h \leq m$: در هر یک از این لایه‌ها b^h گره داریم. این گره‌ها در صورتی خارج می‌شوند که یا یکی از اجداد گره هدف باشند (چون در هر لایه بالاتر از هدف، دقیقاً یکی از گره‌ها جد گره هدف است). پس احتمال اینکه یک گره تصادفی در لایه h جد هدف باشد برابر با $\frac{1}{b^h}$ است. و یا سمت چپ جد گره هدف بوده‌اند و زودتر از آن از fringe خارج شده‌اند (به احتمال $(\frac{1}{b^h})(1 - \frac{1}{b^h})$). اگر این احتمال را برای تمام این گره‌ها جمع بزنیم خواهیم داشت:

$$\sum_{h=0}^m b^h \left(\frac{1}{b^h} + \frac{1}{b^h} \left(1 - \frac{1}{b^h} \right) \right) = \frac{m+1}{2} + \frac{b^{m+1} - 1}{2(b-1)}$$

$h > m$: هر گرهی که در این لایه‌ها قرار دارد، یک جد در عمق m دارد، که این جد به احتمال برابر یکی از b^m گره عمق مورد نظر است. این گره در صورتی پیش از هدف از fringe خارج می‌شود که جد آن در عمق m سمت چپ هدف قرار داشته باشد (به احتمال $(\frac{1}{b^m})(1 - \frac{1}{b^m})$). این احتمال‌ها را جمع می‌زنیم:

$$\sum_{h=m+1}^d b^h \left(\frac{1}{b^h} \right) \left(1 - \frac{1}{b^m} \right) = \frac{b(b^m - 1)(b^{d-m} - 1)}{2(b-1)}$$

این دو مقدار را از نظر اردر جمع می‌زنیم:

$$O\left(\frac{m+1}{2} + \frac{b^{m+1} - 1}{2(b-1)}\right) + O\left(\frac{b(b^m - 1)(b^{d-m} - 1)}{2(b-1)}\right) = O(m + b^m + b^d)$$

$$\xrightarrow{m \leq d} O(b^d)$$

می‌دانیم که BFS ابتدا تمام گره‌های لایه‌های بالاتر را از fringe خارج می‌کند ($\frac{b^m - 1}{b - 1}$ عضو). در لایه به عمق m ، b^m گره داریم. فرض می‌کنیم گره هدف با احتمال برابر یکی از این گره‌ها باشد، امید ریاضی تعداد گره‌هایی که از لایه آخر خارج می‌شوند تا آخرین آنها گره هدف باشد

را محاسبه می‌کنیم:

$$\begin{aligned}\frac{1}{b^m} + \frac{2}{b^m} + \dots + \frac{b^m}{b^m} &= \frac{1}{b^m}(1 + 2 + \dots + b^m) \\ &= \frac{1}{b^m}\left(\frac{b^m(b^m + 1)}{2}\right) = \frac{b^m + 1}{2}\end{aligned}$$

که از $O(b^m)$ است. تعداد گره‌های خارج شده در لایه‌های بالاتر هم از $O(b^{m-1})$ بود؛ پس در کل $O(b^m)$ گره از fringe خارج می‌شوند تا به گره هدف برسیم.

اگر هدف در لایه آخر نباشد ($m < d$)، الگوریتم BFS به طور متوسط عملکرد بهتری نسبت به DFS دارد. البته بر اساس قسمت‌های قبل می‌توان به طور شهودی گفت که اگر هدف سمت چپ درخت باشد، DFS عملکرد بهتری دارد و هر چه هدف به سمت راست حرکت کند، عملکرد DFS کاهش می‌یابد تا زمانی که عملکرد BFS بهتر از آن شود.

(ج) بر اساس امید ریاضی‌هایی که در قسمت قبل محاسبه کردیم، در این مثال تا زمانی که هدف در لایه آخر نباشد، عملکرد BFS بهتر است؛ اگر هدف در لایه آخر باشد، تعداد گره‌های بسط داده شده در هر دو الگوریتم از یک مرتبه است.

سوال ۳:

(آ) مسئله را به این صورت مدل سازی می کنیم:

- **حالت ها:** هر جایگشت دلخواه از n عدد مورد نظر یکی از حالت های مسئله هست و در کل $n!$ حالت داریم.
- **کنش ها:** طبق صورت سوال، جابه جایی هر دو عضو همسایه، یک کنش است. در قسمت های بعد، هر کنش را با $swap(i)$ نشان می دهیم که به معنای جابجایی اعضای i ام و $i + 1$ ام است.
- **حالت اولیه:** همان ترتیبی از اعداد که در ابتدا به ما داده شده حالت اولیه مسئله را تشکیل می دهد؛ هر یک از $n!$ جایگشت ممکن می تواند حالت اولیه ما باشد.
- **حالت نهایی:** جایگشتی که n عدد مورد نظر به طور صعودی مرتب شده باشند.
- **تابع اول:** برای جایگشت π ، تابع fitness زیر را تعریف می کنیم:

$$f_1(\pi) = |\{i \mid \pi_i = i\}|$$

به بیان ساده تر، تابع f_1 تعداد اعدادی که در جایگشت فعلی در مکان حقیقی خود قرار دارند را نشان می دهد؛ بدیهی است که به ازای حالت کامل π^* ، داریم $f_1(\pi^*) = n$ و به ازای هر جایگشت دیگر، $f_1 < n$.

اثبات. می خواهیم نشان دهیم که به ازای هر مسئله با $n \geq 3$ تابع f_1 حداقل یک نقطه بهینه محلی غیر سراسری دارد. جایگشت زیر را در نظر بگیرید:

$$\pi' = (3, 2, 1, 4, 5, \dots, n)$$

روشن است که:

$$f_1(\pi') = 1 + (n - 3) = n - 2 \Rightarrow f_1(\pi') < n$$

پس π' نقطه بهینه سراسری نیست. (I)
اکنون می توانیم از این حالت را می توان به ۴ دسته تقسیم کرد؛ مقدار جدید f_1 به ازای آنها را حساب می کنیم:

$$\begin{aligned} \pi' &\xrightarrow{swap(1)} \pi'_{new} = (2, 3, 1, 4, 5, \dots, n) \Rightarrow f_1(\pi'_{new}) = n - 3 \\ \pi' &\xrightarrow{swap(2)} \pi'_{new} = (3, 1, 2, 4, 5, \dots, n) \Rightarrow f_1(\pi'_{new}) = n - 3 \\ \pi' &\xrightarrow{swap(3)} \pi'_{new} = (3, 2, 4, 1, 5, \dots, n) \Rightarrow f_1(\pi'_{new}) = n - 3 \\ \pi' &\xrightarrow{swap(i)_{i \geq 3}} \pi'_{new} = (3, 2, 1, \dots) \Rightarrow f_1(\pi'_{new}) = n - 4 \end{aligned}$$

همان طور که مشاهده کردیم، تمام همسایه های π' مقدار fitness کمتری نسبت به آن دارند. (II)
از (I) و (II) نتیجه می گیریم که جایگشت π' یک نقطه بهینه محلی غیر سراسری برای f_1 است. از آنجا که π' را برای هر مسئله $n \geq 3$ دلخواه در نظر گرفتیم، پس تابع f_1 همواره حداقل یک نقطه بهینه محلی غیر سراسری دارد. $\square$

- اگر $inv(\pi)$ تعداد نابجایی های^۵ جایگشت π باشد، تابع fitness جدید را به این شکل تعریف می کنیم:

$$f_2(\pi) = -inv(\pi)$$

روشن است که برای حالت کامل π^* داریم $f_2(\pi) = 0$ و به ازای هر جایگشت دیگر $f_2 < 0$.

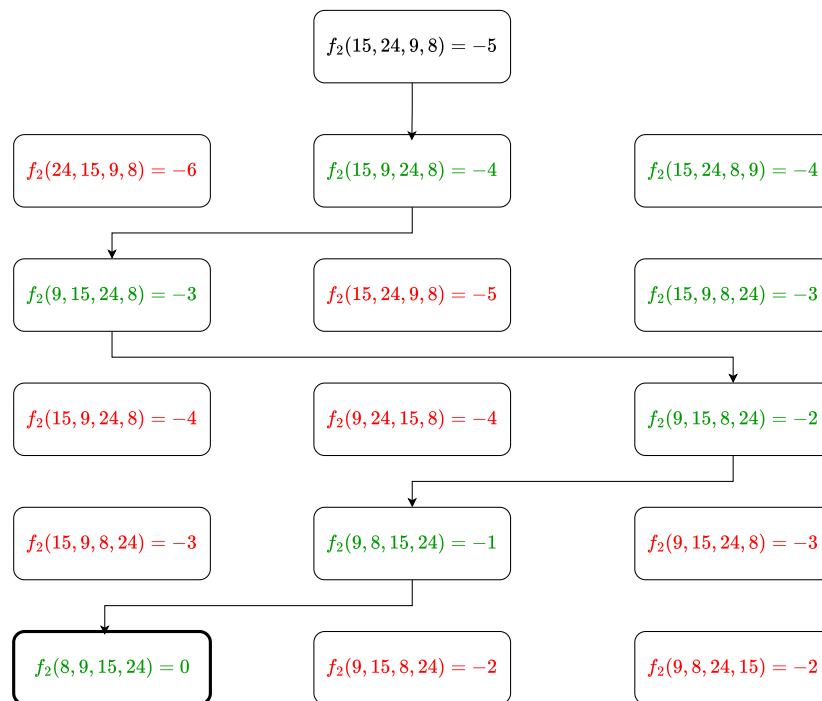
^۵ هر زوج $j > i$ که $\pi_i < \pi_j$ باشد را یک نابجایی یا inversion می نامیم

اثبات. نشان می‌دهیم که به ازای هیچ $n \geq 3$ ، تابع f_2 نقطه بهینه محلی غیرسراسری ندارد. فرض کنید $\pi \neq \pi^*$ یک جایشتگ مسئله باشد. ادعا می‌کنیم $1 \leq i < n$ وجود دارد به طوری که $\pi_{i+1} < \pi_i$ ؛ اگر چنین نباشد، آنگاه می‌توان نتیجه گرفت که $\pi_1 < \pi_2 < \dots < \pi_n$ که در این صورت شرط $\pi \neq \pi^*$ نقض می‌شود. پس $(i, i+1)$ یک نابجایی در جایشتگ π است. حال اگر اکشن $swap(i)$ را انجام دهیم، این نابجایی حذف می‌شود. از طرف دیگر با این اکشن، هیچ تغییری در نابجایی‌های به شکل $(i, j)_{j \neq i+1}$ و $(j, i+1)_{j \neq i}$ به وجود نمی‌آید. پس داریم:

$$f_2(\pi_{new}) = f_2(\pi) + 1$$

پس هیچ جایشتگ $\pi \neq \pi^*$ نمی‌تواند یک نقطه بهینه محلی باشد. پس به ازای این تابع fitness،
 $\square$ نقطه بهینه محلی غیرسراسری نداریم.

(ج) در هر مرحله، به ترتیب اکشن‌های $swap(1)$ ، $swap(2)$ و $swap(3)$ را انجام می‌دهیم تا همسایه‌های حالت فعلی را بسازیم؛ سپس یکی از همسایه‌ها با بالاترین میزان fitness را انتخاب می‌کنیم. اگر هیچ همسایه‌ای fitness بهتر از حالت فعلی نداشت، الگوریتم خاتمه می‌یابد. اگر هم چند همسایه بالاترین میزان fitness را داشتند، اولویت با همسایه‌ای هست که زودتر ساخته شده.



به علت ویژگی involution اکشن‌ها در این مسئله که اگر یک اکشن را دو بار به صورت متوالی روی یک حالت انجام دهیم دوباره به حالت اولیه برمی‌گردیم، همواره یکی از همسایه‌های تولید شده fitness بدتری نسبت به حالت فعلی دارد و می‌توانستیم اصلاً آن را تولید نکنیم. همچنین همان طور که در قسمت قبل اشاره شد، هر اکشن میزان fitness را تنها یک واحد تغییر می‌دهد، و از طرفی نقطه بهینه محلی غیرسراسری هم نداریم. پس می‌توانیم در هر مرحله یک نابجایی بین دو عضو متوالی را پیدا کرده و آنها را swap کنیم. در این صورت دیگر نیازی به تولید همه همسایه‌ها نبود.

(د) الگوریتم Simulated Annealing را اجرا می‌کنیم:

• مرحله ۱ ($t = 1$):

- حالت فعلی: $f_2(\pi) = -5 \mid T = 5 \mid \pi = (15, 24, 9, 8)$
- همسایه ($swap(1)$): $f_2(\pi') = -6 \mid \pi' = (24, 15, 9, 8)$
- * $\Delta E = f_2(\pi') - f_2(\pi) = (-6) - (-5) = -1$
- * محاسبه احتمال پذیرش: $P = e^{\Delta E/T} = e^{-1/5} \approx 0.818$
- * عدد تصادفی مصرف شده: $r = 0.27$
- * نتیجه: چون $r \leq P$ ، این همسایه پذیرفته می‌شود.

• مرحله ۲ ($t = 2$):

- حالت فعلی: $f_2(\pi) = -6 \mid T = 4 \mid \pi = (24, 15, 9, 8)$
- همسایه ($swap(1)$): $f_2(\pi') = -5 \mid \pi' = (15, 24, 9, 8)$
- * $\Delta E = f_2(\pi') - f_2(\pi) = (-5) - (-6) = 1$
- * نتیجه: این همسایه بهتر از حالت فعلی بوده و انتخاب می‌شود.

• مرحله ۳ ($t = 3$):

- حالت فعلی: $f_2(\pi) = -5 \mid T = 3 \mid \pi = (15, 24, 9, 8)$
- همسایه ($swap(1)$): $f_2(\pi') = -6 \mid \pi' = (24, 15, 9, 8)$
- * $\Delta E = f_2(\pi') - f_2(\pi) = (-6) - (-5) = -1$
- * محاسبه احتمال پذیرش: $P = e^{\Delta E/T} = e^{-1/3} \approx 0.716$
- * عدد تصادفی مصرف شده: $r = 0.66$
- * نتیجه: چون $r \leq P$ ، این همسایه پذیرفته می‌شود.

• مرحله ۴ ($t = 4$):

- حالت فعلی: $f_2(\pi) = -6 \mid T = 2 \mid \pi = (24, 15, 9, 8)$
- همسایه ($swap(1)$): $f_2(\pi') = -5 \mid \pi' = (15, 24, 9, 8)$
- * $\Delta E = f_2(\pi') - f_2(\pi) = (-5) - (-6) = 1$
- * نتیجه: این همسایه بهتر از حالت فعلی بوده و انتخاب می‌شود.

• مرحله ۵ ($t = 5$):

- حالت فعلی: $f_2(\pi) = -5 \mid T = 1 \mid \pi = (15, 24, 9, 8)$
- همسایه ($swap(1)$): $f_2(\pi') = -6 \mid \pi' = (24, 15, 9, 8)$
- * $\Delta E = f_2(\pi') - f_2(\pi) = (-6) - (-5) = -1$
- * محاسبه احتمال پذیرش: $P = e^{\Delta E/T} = e^{-1/1} \approx 0.367$
- * عدد تصادفی مصرف شده: $r = 0.91$

* نتیجه: چون $r > P$ ، این همسایه پذیرفته نمی‌شود و در مرحله بعد، همسایه بعدی را بررسی می‌کنیم.

• مرحله ۶ ($t = 6$):

- حالت فعلی: $f_v(\pi) = -5 \mid T = \bullet \mid \pi = (15, 24, 9, 8)$
- همسایه ($swap(2)$): $f_v(\pi') = -4 \mid \pi' = (15, 9, 24, 8)$
- $\Delta E = f_v(\pi') - f_v(\pi) = (-4) - (-5) = 1$ *
- * نتیجه: این همسایه بهتر است و انتخاب می‌شود.

• مرحله ۷ ($t = 7$):

- حالت فعلی: $f_v(\pi) = -4 \mid T = \bullet \mid \pi = (15, 9, 24, 8)$
- همسایه ($swap(1)$): $f_v(\pi') = -3 \mid \pi' = (9, 15, 24, 8)$
- $\Delta E = f_v(\pi') - f_v(\pi) = (-3) - (-4) = 1$ *
- * نتیجه: این همسایه بهتر است و انتخاب می‌شود.

• مرحله ۸ ($t = 8$):

- حالت فعلی: $f_v(\pi) = -3 \mid T = \bullet \mid \pi = (9, 15, 24, 8)$
- همسایه ($swap(1)$): $f_v(\pi') = -4 \mid \pi' = (15, 9, 24, 8)$
- $\Delta E = f_v(\pi') - f_v(\pi) = (-4) - (-3) = -1$ *
- * نتیجه: در این مرحله $T = \bullet$ بنابراین همسایه بدتر انتخاب نمی‌شود.

الگوریتم بدون رسیدن به حالت هدف، خاتمه یافت.

سوال ۴:

در هر گام، مسیرهای فعلی موجود در fringe و مسیری که در آن گام از خارج می‌شود را مشخص می‌کنیم.

• جستجوی BFS:

step	Fringe
۱	(S)
۲	(S), (SA), (SB), (SG)
۳	(S), (SA), (SB), (SG), (SAC), (SAD)
۴	(S), (SA), (SB), (SG), (SAC), (SAD), (SBD), (SBE)

خروجی الگوریتم، مسیر SG هست.

• جستجوی DFS:

step	Fringe
۱	(S)
۲	(S), (SG), (SB), (SA)
۳	(S), (SG), (SB), (SA), (SAD), (SAC)
۴	(S), (SG), (SB), (SA), (SAD), (SAC), (SACG)

خروجی الگوریتم، مسیر SACG است.

• جستجوی UCS:

step	Fringe
۱	(S0)
۲	(S0), (SA2), (SB1), (SG9)
۳	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5)
۴	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5), (SAC4), (SAD5)
۵	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5), (SAC4), (SAD5), (SBDG7)
۶	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5), (SAC4), (SAD5), (SBDG7), (SACG8)
۷	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5), (SAC4), (SAD5), (SBDG7), (SACG8), (SADG9)
۸	(S0), (SA2), (SB1), (SG9), (SBD3), (SBE5), (SAC4), (SAD5), (SBDG7), (SACG8), (SADG9)

الگوریتم UCS، مسیر بهینه $SBDG$ را انتخاب می‌کند.

• جستجوی Greedy:

step	Fringe
۱	(S6)
۲	(S6), (SA0), (SB6), (SG0)
۳	(S6), (SA0), (SB6), (SG0), (SAC4), (SAD1)

الگوریتم Greedy به سرعت مسیر SG را انتخاب می‌کند، که لزوماً انتخاب بهینه نیست.

• جستجوی درختی A^* :

step	Fringe
۱	(S 0+6)
۲	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0)
۳	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1)
۴	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0)
۵	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0), (SBD 3+1), (SBE 5+10)
۶	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0), (SBD 3+1), (SBE 5+10), (SBDG 7+0)

به علت قابل قبول بودن تابع هیوریستیک، همان طور که انتظار داشتیم الگوریتم A^* مسیر بهینه را پیدا کرد. این الگوریتم از UCS سریع تر، و از الگوریتم Greedy کندتر عمل کرد.

• جستجوی گرافی A^* :

step	Fringe	Closed
۱	(S 0+6)	{ }
۲	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0)	{S}
۳	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1)	{S, A}
۴	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0)	{S, A, D}
۵	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0), (SBD 3+1), (SBE 5+10)	{S, A, D, B}
۶	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0), (SBD 3+1), (SBE 5+10)	{S, A, D, B}
۷	(S 0+6), (SA 2+0), (SB 1+6), (SG 9+0), (SAC 4+4), (SAD 5+1), (SADG 9+0), (SBD 3+1), (SBE 5+10), (SACG 8+0)	{S, A, D, B, C}

چون D از قبل در Closed Set بود، الگوریتم مسیر SBD را بسط نداد، و در نهایت یک مسیر غیربهینه را پیدا کرد.

سوال ۵:

(آ) طبق تعریف تابع fitness در این مسئله داریم:

$$f(x_1) = 2(5) + (2) + 3(3) - (6) + 2(4) + (1) = 24$$

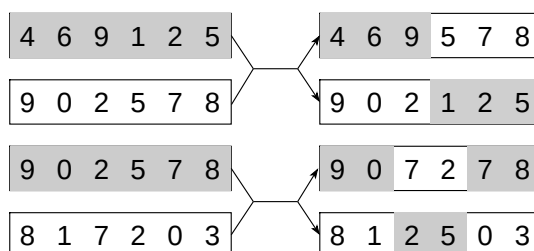
$$f(x_2) = 2(8) + (1) + 3(7) - (2) + 2(0) + (3) = 39$$

$$f(x_3) = 2(4) + (6) + 3(9) - (1) + 2(2) + (5) = 49$$

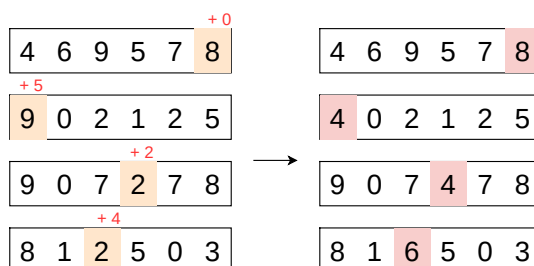
$$f(x_4) = 2(9) + (0) + 3(2) - (5) + 2(7) + (8) = 41$$

بر اساس مقادیر fitness محاسبه شده، مشاهده می‌شود که x_3 بهترین عضو در جمعیت اولیه است.

(ب) عملیات تقاطع (Crossover) به این شکل است:



(ج) عملیات جهش را مطابق با صورت سوال انجام می‌دهیم:



(د) ابتدا fitness جمعیت جدید را محاسبه می‌کنیم:

$$f(x'_1) = 2(4) + (6) + 3(9) - (5) + 2(7) + (8) = 58$$

$$f(x'_2) = 2(4) + (0) + 3(2) - (1) + 2(2) + (5) = 22$$

$$f(x'_3) = 2(9) + (0) + 3(7) - (4) + 2(7) + (8) = 57$$

$$f(x'_4) = 2(8) + (1) + 3(6) - (5) + 2(0) + (3) = 33$$

میانگین جمعیت اولیه و جمعیت جدید را محاسبه می‌کنیم:

$$\overline{f(x)} = \frac{24 + 39 + 49 + 41}{4} = 38.25$$

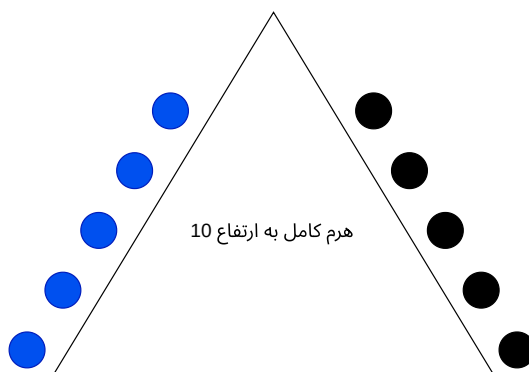
$$\overline{f(x')} = \frac{58 + 22 + 57 + 33}{4} = 42.5$$

همان طور که انتظار داشتیم، مشاهده می‌شود که میانگین fitness در جمعیت جدید بیشتر از جمعیت اولیه است.

سوال ۶:

(آ) مسئله را به این شکل تعریف می‌کنیم:

- **حالت‌ها:** می‌توانیم لیوان‌ها را در حالت کامل شماره‌گذاری کنیم (مثلاً با شروع از ردیف پایین، و در هر ردیف از چپ به راست). در این صورت هر حالت، مجموعه لیوان‌هایی است که باقی مانده‌اند؛ بدیهی است که هر حالت زیرمجموعه حالت اولیه محسوب می‌شود.
 - **کنش‌ها:** ضربه زدن به هر لیوانی در حالت فعلی که حداکثر یک لیوان به طور مستقیم روی آن قرار گرفته، و انداختن آن لیوان و تمام لیوان‌هایی که به طور مستقیم و غیرمستقیم بالای آن قرار گرفتند. (با توجه به محدودیت اعمال شده در انتخاب لیوان، نمی‌توانیم به لیوان‌هایی که دو لیوان روی آنها قرار گرفته ضربه بزنیم).
 - **هزینه کنش‌ها:** هزینه تمام کنش‌ها را مساوی و برابر با ۱ در نظر می‌گیریم.
 - **حالت(های) هدف:** هر مجموعه‌ای از لیوان‌ها که تعداد لیوان‌های ریخته شده در آن بیشتر از نصف کل لیوان‌های حالت اولیه باشد، حالت هدف محسوب می‌شود. (با این تعریف بعضی از حالت‌های هدف، ممکن نیستند؛ به عنوان مثال حالت $\{1, 2, 3, 4, 5, 7, 8, 9, 10\}$ را برای مسئله ترسیم شده در صورت سوال در نظر بگیرید. این حالت از دو نظر مشکل دارد: اول اینکه در حالت اولیه دو لیوان به طور مستقیم روی لیوان ۶ قرار گرفته‌اند و نمی‌توانیم به آن ضربه بزنیم. همچنین اگر فرض کنیم به این لیوان ضربه زده باشیم، لیوان‌های ۸، ۹ و ۱۰ که به طور مستقیم و غیرمستقیم روی آن قرار دارند هم باید حذف می‌شدند. پس این حالت هدف قابل دستیابی نیست).
- (ب) اگر بخواهیم در هر مرحله تعداد لیوان‌هایی که ریخته می‌شوند را بیشینه کنیم، در حالت اولیه باید به راست‌ترین یا چپ‌ترین لیوان ردیف پایین ضربه بزنیم. هر مورد را جداگانه بررسی می‌کنیم:
- **قابل قبول بودن:** با ارائه مثال نقض، نشان می‌دهیم که تابع h قابل قبول نیست. فرض کنید در ابتدا یک هرم کامل با ارتفاع ۱۴ داریم. این هرم در کل $105 = \frac{14 \times 15}{2}$ لیوان دارد. پس باید حداقل ۵۳ لیوان بیفتند. حال فرض کنید تعدادی از لیوان‌ها را حذف کردیم و در نهایت به حالت زیر رسیدیم:



بهترین انتخاب در این حالت، انتخاب یکی از دو لیوان کناری در پایین‌ترین لایه هست، که در این صورت ۵ لیوان فرومی‌ریزند. پس $f(c) = 5$. در این حالت $105 - 40 = 65$. $\max_{c \in \text{cups targetable}(S)} f(c) = 5$. $105 - 40 = 65$. $\frac{10 \times 11}{2} + 10 = 65$. لیوان داریم، پس $105 - 65 = 40$ لیوان تا الان افتاده‌اند و باید $40 - 53 = 13$ لیوان دیگر را بیندازیم. پس:

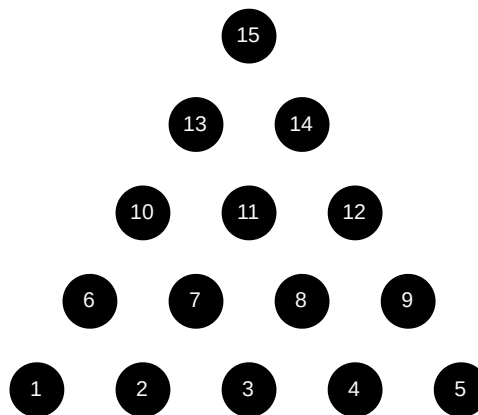
$$h(S) = \frac{13}{5} = 2/6$$

حال فرض کنید یکی از دو لیوان مورد نظر را هدف قرار دهیم. سپس، می‌توانیم یکی از لیوان‌های کناری در لایه پایین هرم ۱۰ تایی را هدف قرار دهیم. در این صورت ۱۰ لیوان می‌افتند؛ در کل ۵۵ لیوان تا الان افتاده‌اند و به حالت هدف رسیده‌ایم.

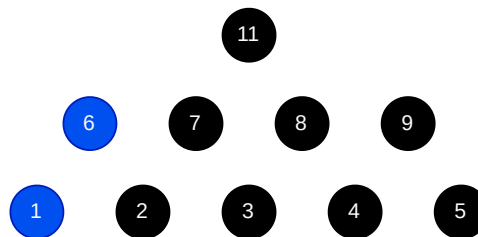
$$h(S) = 2/6 \not\leq h^*(S) = 2$$

پس تابع h قابل قبول نیست.

- **یک‌نوا بودن:** با یک مثال نقض، نشان می‌دهیم که تابع مورد نظر یک‌نوا نیست. فرض کنید در ابتدا یک هرم کامل با ارتفاع ۵ داریم:



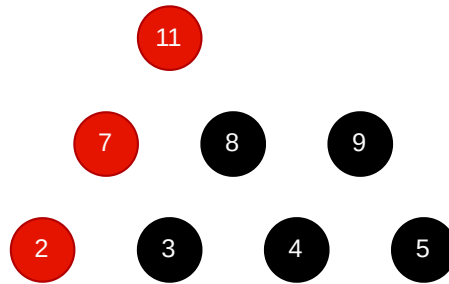
در هرم کامل، $15 = \frac{5 \times 6}{2}$ لیوان داریم و باید حداقل ۸ لیوان فرو بریزند. فرض کنید ۵ تا از لیوان‌های بالا حذف شده‌اند و به این حالت رسیدیم:



مقدار $\max_{c \in \text{cups targetable}(S)} f(c)$ در این حالت، به ازای لیوان شماره ۱ (یا لیوان شماره ۵) رخ می‌دهد که با ضربه زدن به آن، دو لیوان فرو می‌ریزند. (توجه کنید که در این حالت نمی‌توانیم لیوان‌های شماره ۲ یا ۴ را هدف قرار دهیم) بنابراین مقدار تابع h در این حالت برابر است با:

$$h(S) = \frac{3}{4}$$

به توپ شماره ۱ ضربه زده (این اکشن هزینه ۱ دارد) و به این حالت می‌رسیم:



اکنون که یکی از لیوان‌های روی لیوان شماره ۲ افتاده، می‌توانیم این لیوان را هدف قرار دهیم که در این صورت ۳ لیوان می‌افتند. پس در این حالت:

$$h(S') = \frac{1}{3}$$

داریم:

$$h(S) - h(S') = \frac{3}{2} - \frac{1}{3} = \frac{7}{6} \not\leq \text{cost}(S \rightarrow S') = 1$$

بنابراین تابع h یک‌نوا نیست.

(ج) تابع اکتشافی h' را به این شکل تعریف می‌کنیم:

$$h'(S) = \begin{cases} 2 & \text{S حالت اولیه باشد} \\ 0 & \text{S حالت هدف باشد} \\ 0.5 & \text{حالت‌های دیگر} \end{cases}$$

برای اثبات قابل قبول بودن h' باید نشان دهیم:

$$\forall S : 0 \leq h'(S) \leq h^*(S)$$

برای حالت‌هایی به جز حالت اولیه، بدیهی است که این رابطه صحیح می‌باشد. اگر ارتفاع هرم n باشد، در کل $\frac{n(n+1)}{2}$ لیوان داریم و باید حداقل $\lfloor \frac{n(n+1)}{4} + 1 \rfloor$ لیوان فرو ببریم. همچنین در هر حالت، حداکثر n لیوان را می‌توانیم بیندازیم، که البته می‌دانیم این کار تنها در یک حرکت ممکن است. پس:

$$h^*(S) \geq \frac{\lfloor \frac{n(n+1)}{4} + 1 \rfloor}{n} > \frac{\frac{n(n+1)}{4}}{n} = \frac{n+1}{4}$$

که S ، حالت اولیه است. با فرض اینکه ارتفاع هرم اولیه حداقل ۱۵ باشد، داریم $h^*(S) \geq 4$ و در نتیجه $h'(S) < h^*(S)$. پس تابع h' قابل قبول است.

برای نشان دادن اینکه این تابع یک‌نوا نیست، کافی است مثالی را تصور کنیم که از حالت اولیه S به حالت S' می‌رویم. (S حالت هدف نیست)

$$h'(S') - h'(S) = 2 - 0.5 = 1.5 \not\leq \text{cost}(S \rightarrow S') = 1$$

پس تابع اکتشافی h' ، یک‌نوا نیست.